Khoa học Dữ liệu

Notebook: Tổng quan Python và xử lý dữ liệu cơ bản
Năm học: 2025-2026

Link jupyter notebook: https://colab.research.google.com/drive/1rl8KzdHThHnjJ6EAjIUvKvnEZq_gLsen?usp=sharing

Link bài tập: https://colab.research.google.com/drive/1UWF4q9HsAFedOToU1YPbIo3SbsDHDGaE?usp=sharing

1. Cài đặt Python:

- Cách 1: Anaconda
  - Tải tại: https://www.anaconda.com/download
  - https://www.anaconda.com/docs/psm/on-prem/latest/user/notebook
- Cách 2: Python chính thức từ python.org
  - Tải tại: https://www.python.org/downloads/

2. Sử dụng Jupyter notebook (Khuyến khích sử dụng)

- Google Colaboratory https://colab.research.google.com/
- Sử dụng Anaconda/Python kết hợp với IDE

3. Tính toán cơ bản với python

3.1 Các phép toán số học

- **Cộng (+):** Dùng để cộng hai số lại với nhau.
- **Trừ (-):** Dùng để lấy số thứ nhất trừ đi số thứ hai.
- **Nhân (*):** Dùng để nhân hai số.
- **Chia (/):** Dùng để chia số thứ nhất cho số thứ hai.
- **Lũy thừa ()**:** Dùng để tính số thứ nhất mũ số thứ hai.

Ví dụ:

- `a + b` sẽ cho ra tổng của a và b.
- `a ** b` sẽ cho ra kết quả của a lũy thừa b.

Các phép toán này là nền tảng cho mọi chương trình Python, giúp bạn thực hiện các phép tính toán học cơ bản.

Các phép toán cơ bản như cộng, trừ, nhân, chia và lũy thừa là nền tảng của mọi chương trình tính toán. Việc thực hành các phép toán này giúp bạn hiểu cách Python xử lý số liệu và là bước đầu tiên để xây dựng các chương trình phức tạp hơn.

In [1]:
```
a = 10
b = 3
type(a)
```

Out[1]:
```
int
```

In [2]:
```
a = input("Nhập a: ")
b = input("Nhập b: ")
print("a + b =", a + b)
```

```
Nhập a: 1
Nhập b: 9
a + b = 19
```

Khi nhập dữ liệu từ bàn phím bằng hàm `input()`, Python mặc định coi dữ liệu là chuỗi. Nếu không chuyển sang kiểu số, phép cộng sẽ thực hiện nối chuỗi thay vì cộng số. Việc chuyển kiểu giúp đảm bảo kết quả đúng với mong muốn khi làm việc với số liệu.

Việc sử dụng hàm `int()` khi nhập dữ liệu giúp đảm bảo giá trị nhập vào là số nguyên, tránh lỗi khi thực hiện các phép toán. Đây là thao tác quan trọng khi xử lý dữ liệu đầu vào để đảm bảo tính chính xác cho các phép tính tiếp theo.

In [3]:
```
a = int(input("Nhập a: "))
b = int(input("Nhập b: "))
print("a + b =", a + b)
```

```
Nhập a: 1
Nhập b: 9
a + b = 10
```

In [4]:
```
print("a + b =", a + b)   # cộng
print("a - b =", a - b)   # trừ
print("a * b =", a * b)   # nhân
print("a / b =", a / b)   # chia
print("a^b =", a ** b)    # lũy thừa
```

```
a + b = 10
a - b = -8
a * b = 9
a / b = 0.1111111111111111
a^b = 1
```

3.2 Các phép toán trên hàm số

- Ở phần trên là một số phép toán cơ bản với Python, để xử lý một số phép toán phức tạp hơn ta cần sử dụng thư viện `Numpy` để thực hiện một số phép toán trên hàm số.

In [5]:
```
import numpy as np   # Import thư viện
```

In [6]:
```
print(np.sqrt(a))
```

```
1.0
```

In [7]:
```
np.abs(b-a)
```

Out[7]:
```
np.int64(8)
```

In [8]:
```
np.sin(a)
```

Out[8]:
```
np.float64(0.8414709848078965)
```

In [9]:
```
print(np.log(a))
```

```
0.0
```

In [10]:
```
print(np.exp(a))
```

```
2.718281828459045
```

In [11]:
```
print(np.exp(-a**2 / 2))
```

```
0.6065306597126334
```

In [12]:
```
print(np.exp(a)/(1+np.exp(a)))
```

```
0.7310585786300049
```

In [13]:
```
from scipy.stats import norm  # Thư viện để tính toán thống kê
norm.pdf((a - 3) / b) / b     # Tính hàm mật độ xác suất tại một giá trị
```

Out[13]:
```
np.float64(0.043245829907971815)
```

3.3 Các phép toán trên tập hợp

In [14]:
```
print('a:', a)
print('b:', b)
```

```
a: 1
b: 9
```

In [15]:
```
print("a = b:", a == b)   # Bằng
print("a != b:", a != b)  # Khác
print("a > b:", a > b)    # Lớn hơn
print("a < b:", a < b)    # Nhỏ hơn
print("a >= b:",a >= b)   # Lớn hơn hoặc bằng
print("a <= b:",a <= b)   # Nhỏ hơn hoặc bằng
```

```
a = b: False
a != b: True
a > b: False
a < b: True
a >= b: False
a <= b: True
```

In [16]:
```
rocks_A = {"granite", "basalt", "limestone"}
rocks_B = {"basalt", "shale", "sandstone"}
```

```
print("Hợp (union):", rocks_A | rocks_B)  # Hợp (union)
print("Giao (intersection):", rocks_A & rocks_B)  # Giao (intersection)
print("Hiệu (difference):", rocks_A - rocks_B)    # Hiệu (difference)
print("Phần khác nhau (symmetric difference):", rocks_A ^ rocks_B)  # Phần khác nhau (symmetr
```

```
Hợp (union): {'limestone', 'granite', 'shale', 'sandstone', 'basalt'}
Giao (intersection): {'basalt'}
Hiệu (difference): {'limestone', 'granite'}
Phần khác nhau (symmetric difference): {'limestone', 'granite', 'shale', 'sandstone'}
```

4. Các kiểu dữ liệu chính trong Python:

- **Array:** Là mảng số học đồng nhất, chỉ chứa một kiểu dữ liệu (thường là số). Hỗ trợ các phép toán vector, ma trận.
- **List:** Danh sách các phần tử, có thể thay đổi, dùng để lưu trữ nhiều giá trị. Ví dụ: `[1, 2, 3]`.
- **Tuple:** Tập hợp các phần tử, giống List nhưng không thể thay đổi. Ví dụ: `(1, 2, 3)`.
- **Dictionary:** Lưu trữ dữ liệu dưới dạng cặp khóa-giá trị. Ví dụ: `{'name': 'Alice', 'age': 25}`.
- **DataFrame:** Cấu trúc dữ liệu dạng bảng, thường dùng trong phân tích dữ liệu với pandas.

4.1 Mảng (Array)

In [17]:
```
arr = np.arange(1, 20 + 1)
arr
```

Out[17]:
```
array([ 1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16, 17,
       18, 19, 20])
```

In [18]:
```
x = np.linspace(1, 20, 20)
x
```

Out[18]:
```
array([ 1.,  2.,  3.,  4.,  5.,  6.,  7.,  8.,  9., 10., 11., 12., 13.,
       14., 15., 16., 17., 18., 19., 20.])
```

Các phép toán cơ bản có thể sử dụng cho các phần tử trên mảng

In [19]:
```
np.abs(arr - 10)
```

Out[19]:
```
array([ 9,  8,  7,  6,  5,  4,  3,  2,  1,  0,  1,  2,  3,  4,  5,  6,  7,
        8,  9, 10])
```

In [20]:
```
np.exp(arr)/(1+np.exp(arr))
```

Out[20]:
```
array([0.73105858, 0.88079708, 0.95257413, 0.98201379, 0.99330715,
       0.99752738, 0.99908895, 0.99966465, 0.99987661, 0.9999546 ,
       0.9999833 , 0.99999386, 0.99999774, 0.99999917, 0.99999969,
       0.99999989, 0.99999996, 0.99999998, 0.99999999, 1.        ])
```

In [21]:
```
print(sum(arr))
```

```
210
```

In [22]:
```
print(np.mean(arr))
```

```
10.5
```

In [23]:
```
print(np.median(arr))
```

```
10.5
```

In [24]:
```
print(np.std(arr))
```

```
5.766281297335398
```

In [25]:
```
print(np.var(arr))
```

```
33.25
```

In [26]:
```
len_arr = len(arr)
```

In [27]:
```
np.arange(1, len_arr + 1) / len_arr
```

Out[27]:
```
array([0.05, 0.1 , 0.15, 0.2 , 0.25, 0.3 , 0.35, 0.4 , 0.45, 0.5 , 0.55,
       0.6 , 0.65, 0.7 , 0.75, 0.8 , 0.85, 0.9 , 0.95, 1.  ])
```

- Với một số phép tính toán gồm nhiều bước và cần gom một đoạn code lại thành một khối có thể tái sử dụng, thay vì phải viết lặp đi lặp lại. Lúc này việc viết hàm sẽ hiệu quả hơn.

In [28]:
```
def ecdf_fun(x):
    x = np.sort(np.array(x))
    n = len(x)
    y = np.arange(1, n + 1) / n
    return y

y = ecdf_fun(arr)
```

4.2 List

List là kiểu dữ liệu linh hoạt, cho phép lưu trữ nhiều giá trị và dễ dàng thao tác như thêm, xóa, sửa. Việc kiểm tra kiểu dữ liệu giúp xác định đúng loại biến đang sử dụng, tránh nhầm lẫn khi xử lý dữ liệu.

In [29]:
```
rocks = ["Granite", "Basalt", 20, "Limestone"]
print(rocks)
```

```
['Granite', 'Basalt', 20, 'Limestone']
```

In [30]:
```
type(rocks)
```

Out[30]:
```
list
```

Việc lấy độ dài của list giúp kiểm tra số lượng phần tử, từ đó có thể kiểm soát vòng lặp, điều kiện hoặc xác định dữ liệu đầu vào có đủ hay không cho các thao tác tiếp theo.

In [31]:
```
len(rocks)
```

Out[31]:
```
4
```

Truy xuất phần tử đầu tiên giúp kiểm tra hoặc sử dụng giá trị khởi đầu của danh sách, thường dùng trong các thuật toán hoặc khi cần lấy thông tin đại diện cho dữ liệu.

In [32]:
```
rocks[0]
```

Out[32]:
```
'Granite'
```

Truy xuất phần tử cuối cùng thường dùng để lấy giá trị mới nhất hoặc kiểm tra kết quả sau các thao tác thêm dữ liệu vào danh sách.

In [33]:
```
rocks[-1]
```

Out[33]:
```
'Limestone'
```

Việc lấy một đoạn của list (slice) giúp xử lý hoặc phân tích một phần dữ liệu, ví dụ như lấy các phần tử đầu để kiểm tra, hoặc chia nhỏ dữ liệu cho các thao tác tiếp theo.

In [34]:
```
rocks[0:3]
```

Out[34]:
```
['Granite', 'Basalt', 20]
```

Thay đổi giá trị phần tử trong list giúp cập nhật thông tin khi dữ liệu ban đầu chưa chính xác hoặc cần điều chỉnh theo yêu cầu xử lý.

In [35]:
```
rocks[0] = 'Sandstone'
```

Hiển thị toàn bộ danh sách sau khi thay đổi giúp kiểm tra lại kết quả thao tác, đảm bảo dữ liệu đã được cập nhật đúng như mong muốn.

In [36]:
```
rocks
```

Out[36]:
```
['Sandstone', 'Basalt', 20, 'Limestone']
```

Phương thức append giúp thêm phần tử mới vào cuối danh sách một cách nhanh chóng, thuận tiện cho việc mở rộng dữ liệu khi cần thiết.

In [37]:
```
rocks.append('Quartzite')
rocks
```

Out[37]:
```
['Sandstone', 'Basalt', 20, 'Limestone', 'Quartzite']
```

4.3 Tuple

Tuple được sử dụng khi bạn cần một tập hợp các giá trị cố định, không thay đổi trong suốt quá trình xử lý. Việc kiểm tra kiểu dữ liệu giúp đảm bảo bạn thao tác đúng với loại biến mong muốn, tránh lỗi khi xử lý dữ liệu.

In [38]:
```
coordinates = (10.5, 20.3)
type(coordinates)
```

Out[38]:
```
tuple
```

4.4 Dictionary

Dictionary cho phép lưu trữ dữ liệu dưới dạng cặp khóa-giá trị, rất thuận tiện khi bạn cần truy xuất thông tin theo tên thuộc tính. Việc kiểm tra kiểu dữ liệu giúp xác định đúng cấu trúc dữ liệu đang sử dụng.

In [39]:
```
sample = {"Depth": 1200, "Density": 2.65, "Porosity": 0.15}
print(sample)
type(sample)
```

```
{'Depth': 1200, 'Density': 2.65, 'Porosity': 0.15}
```

Out[39]:
```
dict
```

Truy xuất giá trị theo khóa giúp lấy thông tin cụ thể một cách nhanh chóng, ví dụ như lấy mật độ mẫu địa chất để phân tích hoặc so sánh.

In [40]:
```
sample['Density']
```

Out[40]:
```
2.65
```

4.5 Dataframe

DataFrame là cấu trúc dữ liệu cho phép lưu trữ và thao tác dữ liệu dạng bảng, rất phù hợp cho phân tích dữ liệu lớn, trực quan và dễ sử dụng với các hàm thống kê, lọc, nhóm.

In [41]:
```
import pandas as pd   # Import thư viện

data = {
    "Depth": [100, 200, 300, 400, 500],       # độ sâu (m)
    "Density": [2.65, 2.70, 2.55, 2.80, 2.60], # mật độ (g/cm3)
    "Porosity": [0.15, 0.10, 0.20, 0.08, 0.18] # độ rỗng (fraction)
}

df = pd.DataFrame(data)
```

In [42]:
```
df
```

Out[42]:

	Depth	Density	Porosity
0	100	2.65	0.15
1	200	2.70	0.10
2	300	2.55	0.20
3	400	2.80	0.08
4	500	2.60	0.18

In [43]:
```
df.head(3)    # Hiển thị 3 dòng đầu
```

Out[43]:

	Depth	Density	Porosity
0	100	2.65	0.15
1	200	2.70	0.10
2	300	2.55	0.20

Việc xem thông tin cột và kiểu dữ liệu giúp xác định chính xác các trường dữ liệu, kiểm tra dữ liệu thiếu và đảm bảo các thao tác phân tích sau này được thực hiện đúng với kiểu dữ liệu mong muốn.

In [44]:
```
df.info()    # thông tin cột, kiểu dữ liệu
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 3 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   Depth     5 non-null      int64
 1   Density   5 non-null      float64
 2   Porosity  5 non-null      float64
dtypes: float64(2), int64(1)
memory usage: 252.0 bytes
```

5. Thêm dữ liệu vào Python

In [45]:
```
from google.colab import drive
import pandas as pd

drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

5.1 Đọc dữ liệu từ file txt

- Sử dụng thư viện `numpy` để đọc file dạng số liệu với hàm `np.loadtxt`.
- Sử dụng thư viện `pandas` để đọc file dạng bảng với hàm `pd.read_csv`.
- Thường cần chỉ rõ đường dẫn file, định dạng phân tách (ví dụ: tab, dấu phẩy), và các tùy chọn như có tiêu đề hay không.
- Sau khi đọc dữ liệu, có thể sử dụng các hàm như `.mean()`, `.describe()`, `.info()` để phân tích nhanh dữ liệu.

Sử dụng numpy để đọc file txt giúp xử lý nhanh các dữ liệu số, đặc biệt khi bạn cần thao tác với mảng lớn và thực hiện các phép toán thống kê, phân tích số liệu.

Ở đây, chúng ta khai thác dữ liệu về trọng lượng của kim cương được lưu trư ở định dạng txt

In [46]:
```
import numpy as np    # Import thư viện
```

In [47]:
```
url_diamond_1 = '/content/drive/MyDrive/course_Data_Science_for_Geoscience/Week 1/Dataset/diar
diamond_dataset = np.loadtxt(url_diamond_1)
diamond_dataset [0:10]
```

Out[47]:
```
array([3.45928685, 1.55249399, 0.60598563, 1.37401453, 0.24024135,
       0.86516509, 0.7054729 , 0.6160537 , 0.48822434, 1.07565959])
```

Pandas cho phép đọc dữ liệu dạng bảng từ file txt một cách linh hoạt, dễ dàng xử lý các định dạng phức tạp, thêm các tùy chọn như phân tách, tiêu đề, rất phù hợp cho phân tích dữ liệu thực tế.

In [48]:
```
diamond_dataset1 = pd.read_csv(url_diamond_1, sep = '\t', header=None)
diamond_dataset1
```

Out[48]:

	0
0	3.459287
1	1.552494
2	0.605986
3	1.374015
4	0.240241
...	...
995	1.526046
996	0.265472
997	3.075434
998	0.539756
999	4.275491

1000 rows × 1 columns

5.2 Đọc dữ liệu từ file csv

Ở đây, chúng ta khai thác dữ liệu về dữ liệu catchments (lưu vực/hydrogeological catchments) trong bối cảnh địa chất – thủy văn được lưu trữ ở định dạng csv với các biến:

- lithology – Thành phần thạch học (loại đá nền trong lưu vực)
- age – Niên đại địa chất của đá nền
- springs – Số lượng suối (springs) trong catchment đó.
- pH – Độ axit/kiềm của nước.
- CaMg – Hàm lượng Ca + Mg (ion kiềm thổ) trong nước (mg/L hoặc mol/L).
- vegetation – Độ phủ thực vật (% hoặc chỉ số).

Hiển thị toàn bộ bảng dữ liệu sau khi đọc giúp xác nhận dữ liệu đã được nạp đúng, kiểm tra cấu trúc và phát hiện lỗi định dạng hoặc thiếu dữ liệu.

In [49]:
```
url_catchments = '/content/drive/MyDrive/course_Data_Science_for_Geoscience/Week 1/Dataset/cat
catchments = pd.read_csv(url_catchments)
catchments.head(5)
```

Out[49]:

	lithology	age	springs	pH	CaMg	vegetation
0	basalt	Cenozoic	0	6.2	0.35	5.8
1	granite	Mesozoic	1	4.4	11.00	28.0
2	basalt	Mesozoic	0	5.6	6.00	12.0
3	sandstone	Cenozoic	6	5.2	1.80	27.0
4	shale	Precambrian	3	4.5	2.30	40.0

5.3 Tiền xử lý dữ liệu

- Tiền xử lý dữ liệu là bước giúp phát hiện loại của dữ liệu, dữ liệu thiếu, dữ liệu bị trùng lặp. Từ đó đưa ra sử dụng các phương pháp để xử lý giúp khai thác dữ liệu tốt hơn

Xem thông tin cột và kiểu dữ liệu giúp bạn xác định các trường dữ liệu quan trọng, kiểm tra dữ liệu thiếu và chuẩn bị cho các thao tác phân tích, trực quan hóa tiếp theo.

In [50]:
```
catchments.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20 entries, 0 to 19
Data columns (total 6 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   lithology   20 non-null     object
 1   age         20 non-null     object
 2   springs     20 non-null     int64
 3   pH          20 non-null     float64
 4   CaMg        20 non-null     float64
 5   vegetation  20 non-null     float64
dtypes: float64(3), int64(1), object(2)
memory usage: 1.1+ KB
```

Kiểm tra giá trị thiếu giúp bạn phát hiện các vấn đề về chất lượng dữ liệu, từ đó quyết định xử lý như loại bỏ, thay thế hoặc bổ sung dữ liệu phù hợp.

In [51]:
```
catchments.isna()
```

Out[51]:

	lithology	age	springs	pH	CaMg	vegetation
0	False	False	False	False	False	False
1	False	False	False	False	False	False
2	False	False	False	False	False	False
3	False	False	False	False	False	False
4	False	False	False	False	False	False
5	False	False	False	False	False	False
6	False	False	False	False	False	False
7	False	False	False	False	False	False
8	False	False	False	False	False	False
9	False	False	False	False	False	False
10	False	False	False	False	False	False
11	False	False	False	False	False	False
12	False	False	False	False	False	False
13	False	False	False	False	False	False
14	False	False	False	False	False	False
15	False	False	False	False	False	False
16	False	False	False	False	False	False
17	False	False	False	False	False	False
18	False	False	False	False	False	False
19	False	False	False	False	False	False

Đếm số lượng giá trị thiếu ở mỗi cột giúp bạn xác định mức độ thiếu hụt dữ liệu, từ đó lựa chọn phương pháp xử lý phù hợp cho từng trường dữ liệu. Nếu xuất hiện dữ liệu bị thiếu, có thể loại bỏ chúng nếu số lượng thiếu không quá nhiều, và sau đó thêm `.reset_index(drop = True)` để tránh sai chỉ mục của dòng dữ liệu

In [52]:
```
catchments.isna().sum()   #catchments.dropna().reset_index(drop = True)
```

Out[52]:

	0
lithology	0
age	0
springs	0
pH	0
CaMg	0
vegetation	0

dtype: int64

Kiểm tra dòng trùng lặp giúp đảm bảo dữ liệu không bị lặp lại, tránh sai lệch khi phân tích thống kê hoặc xây dựng mô hình dự báo. Tương tự với việc loại bỏ các dữ liệu thiếu, có thể loại bỏ chúng nhưng cần thêm thêm **.reset_index(drop = True)** để tránh sai chỉ mục của dòng dữ liệu

In [53]:
```
catchments.duplicated().sum()   #catchments.drop_duplicates().reset_index(drop = True)
```

Out[53]:
```
np.int64(0)
```

5.4 Tính toán giá trị tổng hợp dữ liệu

- Một số hàm cơ bản trong Python:

Sử dụng một số hàm cơ bản giúp đánh giá xu hướng chung của dữ liệu, là chỉ số quan trọng trong phân tích thống kê và so sánh các tập dữ liệu khác nhau.

In [54]:
```
np.mean(diamond_dataset)
```

Out[54]:
```
np.float64(1.3718040182631102)
```

In [55]:
```
np.median(diamond_dataset)
```

Out[55]:
```
np.float64(0.6796918143203134)
```

In [56]:
```
np.std(diamond_dataset)
```

Out[56]:
```
np.float64(2.210628264684684)
```

In [57]:
```
ecdf_fun(diamond_dataset)   # Sử dụng hàm thay vì làm từng bước
```

Out[57]:
```
array([0.001, 0.002, 0.003, 0.004, 0.005, 0.006, 0.007, 0.008, 0.009,
       0.01 , 0.011, 0.012, 0.013, 0.014, 0.015, 0.016, 0.017, 0.018,
       0.019, 0.02 , 0.021, 0.022, 0.023, 0.024, 0.025, 0.026, 0.027,
       0.028, 0.029, 0.03 , 0.031, 0.032, 0.033, 0.034, 0.035, 0.036,
       0.037, 0.038, 0.039, 0.04 , 0.041, 0.042, 0.043, 0.044, 0.045,
       0.046, 0.047, 0.048, 0.049, 0.05 , 0.051, 0.052, 0.053, 0.054,
       0.055, 0.056, 0.057, 0.058, 0.059, 0.06 , 0.061, 0.062, 0.063,
       0.064, 0.065, 0.066, 0.067, 0.068, 0.069, 0.07 , 0.071, 0.072,
       0.073, 0.074, 0.075, 0.076, 0.077, 0.078, 0.079, 0.08 , 0.081,
       0.082, 0.083, 0.084, 0.085, 0.086, 0.087, 0.088, 0.089, 0.09 ,
       0.091, 0.092, 0.093, 0.094, 0.095, 0.096, 0.097, 0.098, 0.099,
       0.1  , 0.101, 0.102, 0.103, 0.104, 0.105, 0.106, 0.107, 0.108,
       0.109, 0.11 , 0.111, 0.112, 0.113, 0.114, 0.115, 0.116, 0.117,
       0.118, 0.119, 0.12 , 0.121, 0.122, 0.123, 0.124, 0.125, 0.126,
       0.127, 0.128, 0.129, 0.13 , 0.131, 0.132, 0.133, 0.134, 0.135,
       0.136, 0.137, 0.138, 0.139, 0.14 , 0.141, 0.142, 0.143, 0.144,
       0.145, 0.146, 0.147, 0.148, 0.149, 0.15 , 0.151, 0.152, 0.153,
       0.154, 0.155, 0.156, 0.157, 0.158, 0.159, 0.16 , 0.161, 0.162,
       0.163, 0.164, 0.165, 0.166, 0.167, 0.168, 0.169, 0.17 , 0.171,
       0.172, 0.173, 0.174, 0.175, 0.176, 0.177, 0.178, 0.179, 0.18 ,
       0.181, 0.182, 0.183, 0.184, 0.185, 0.186, 0.187, 0.188, 0.189,
       0.19 , 0.191, 0.192, 0.193, 0.194, 0.195, 0.196, 0.197, 0.198,
       0.199, 0.2  , 0.201, 0.202, 0.203, 0.204, 0.205, 0.206, 0.207,
       0.208, 0.209, 0.21 , 0.211, 0.212, 0.213, 0.214, 0.215, 0.216,
       0.217, 0.218, 0.219, 0.22 , 0.221, 0.222, 0.223, 0.224, 0.225,
       0.226, 0.227, 0.228, 0.229, 0.23 , 0.231, 0.232, 0.233, 0.234,
       0.235, 0.236, 0.237, 0.238, 0.239, 0.24 , 0.241, 0.242, 0.243,
       0.244, 0.245, 0.246, 0.247, 0.248, 0.249, 0.25 , 0.251, 0.252,
       0.253, 0.254, 0.255, 0.256, 0.257, 0.258, 0.259, 0.26 , 0.261,
       0.262, 0.263, 0.264, 0.265, 0.266, 0.267, 0.268, 0.269, 0.27 ,
       0.271, 0.272, 0.273, 0.274, 0.275, 0.276, 0.277, 0.278, 0.279,
       0.28 , 0.281, 0.282, 0.283, 0.284, 0.285, 0.286, 0.287, 0.288,
       0.289, 0.29 , 0.291, 0.292, 0.293, 0.294, 0.295, 0.296, 0.297,
       0.298, 0.299, 0.3  , 0.301, 0.302, 0.303, 0.304, 0.305, 0.306,
       0.307, 0.308, 0.309, 0.31 , 0.311, 0.312, 0.313, 0.314, 0.315,
       0.316, 0.317, 0.318, 0.319, 0.32 , 0.321, 0.322, 0.323, 0.324,
       0.325, 0.326, 0.327, 0.328, 0.329, 0.33 , 0.331, 0.332, 0.333,
       0.334, 0.335, 0.336, 0.337, 0.338, 0.339, 0.34 , 0.341, 0.342,
       0.343, 0.344, 0.345, 0.346, 0.347, 0.348, 0.349, 0.35 , 0.351,
       0.352, 0.353, 0.354, 0.355, 0.356, 0.357, 0.358, 0.359, 0.36 ,
       0.361, 0.362, 0.363, 0.364, 0.365, 0.366, 0.367, 0.368, 0.369,
       0.37 , 0.371, 0.372, 0.373, 0.374, 0.375, 0.376, 0.377, 0.378,
       0.379, 0.38 , 0.381, 0.382, 0.383, 0.384, 0.385, 0.386, 0.387,
       0.388, 0.389, 0.39 , 0.391, 0.392, 0.393, 0.394, 0.395, 0.396,
       0.397, 0.398, 0.399, 0.4  , 0.401, 0.402, 0.403, 0.404, 0.405,
       0.406, 0.407, 0.408, 0.409, 0.41 , 0.411, 0.412, 0.413, 0.414,
       0.415, 0.416, 0.417, 0.418, 0.419, 0.42 , 0.421, 0.422, 0.423,
       0.424, 0.425, 0.426, 0.427, 0.428, 0.429, 0.43 , 0.431, 0.432,
       0.433, 0.434, 0.435, 0.436, 0.437, 0.438, 0.439, 0.44 , 0.441,
       0.442, 0.443, 0.444, 0.445, 0.446, 0.447, 0.448, 0.449, 0.45 ,
       0.451, 0.452, 0.453, 0.454, 0.455, 0.456, 0.457, 0.458, 0.459,
       0.46 , 0.461, 0.462, 0.463, 0.464, 0.465, 0.466, 0.467, 0.468,
       0.469, 0.47 , 0.471, 0.472, 0.473, 0.474, 0.475, 0.476, 0.477,
       0.478, 0.479, 0.48 , 0.481, 0.482, 0.483, 0.484, 0.485, 0.486,
       0.487, 0.488, 0.489, 0.49 , 0.491, 0.492, 0.493, 0.494, 0.495,
       0.496, 0.497, 0.498, 0.499, 0.5  , 0.501, 0.502, 0.503, 0.504,
       0.505, 0.506, 0.507, 0.508, 0.509, 0.51 , 0.511, 0.512, 0.513,
       0.514, 0.515, 0.516, 0.517, 0.518, 0.519, 0.52 , 0.521, 0.522,
       0.523, 0.524, 0.525, 0.526, 0.527, 0.528, 0.529, 0.53 , 0.531,
       0.532, 0.533, 0.534, 0.535, 0.536, 0.537, 0.538, 0.539, 0.54 ,
```

```
0.541, 0.542, 0.543, 0.544, 0.545, 0.546, 0.547, 0.548, 0.549,
0.55 , 0.551, 0.552, 0.553, 0.554, 0.555, 0.556, 0.557, 0.558,
0.559, 0.56 , 0.561, 0.562, 0.563, 0.564, 0.565, 0.566, 0.567,
0.568, 0.569, 0.57 , 0.571, 0.572, 0.573, 0.574, 0.575, 0.576,
0.577, 0.578, 0.579, 0.58 , 0.581, 0.582, 0.583, 0.584, 0.585,
0.586, 0.587, 0.588, 0.589, 0.59 , 0.591, 0.592, 0.593, 0.594,
0.595, 0.596, 0.597, 0.598, 0.599, 0.6  , 0.601, 0.602, 0.603,
0.604, 0.605, 0.606, 0.607, 0.608, 0.609, 0.61 , 0.611, 0.612,
0.613, 0.614, 0.615, 0.616, 0.617, 0.618, 0.619, 0.62 , 0.621,
0.622, 0.623, 0.624, 0.625, 0.626, 0.627, 0.628, 0.629, 0.63 ,
0.631, 0.632, 0.633, 0.634, 0.635, 0.636, 0.637, 0.638, 0.639,
0.64 , 0.641, 0.642, 0.643, 0.644, 0.645, 0.646, 0.647, 0.648,
0.649, 0.65 , 0.651, 0.652, 0.653, 0.654, 0.655, 0.656, 0.657,
0.658, 0.659, 0.66 , 0.661, 0.662, 0.663, 0.664, 0.665, 0.666,
0.667, 0.668, 0.669, 0.67 , 0.671, 0.672, 0.673, 0.674, 0.675,
0.676, 0.677, 0.678, 0.679, 0.68 , 0.681, 0.682, 0.683, 0.684,
0.685, 0.686, 0.687, 0.688, 0.689, 0.69 , 0.691, 0.692, 0.693,
0.694, 0.695, 0.696, 0.697, 0.698, 0.699, 0.7  , 0.701, 0.702,
0.703, 0.704, 0.705, 0.706, 0.707, 0.708, 0.709, 0.71 , 0.711,
0.712, 0.713, 0.714, 0.715, 0.716, 0.717, 0.718, 0.719, 0.72 ,
0.721, 0.722, 0.723, 0.724, 0.725, 0.726, 0.727, 0.728, 0.729,
0.73 , 0.731, 0.732, 0.733, 0.734, 0.735, 0.736, 0.737, 0.738,
0.739, 0.74 , 0.741, 0.742, 0.743, 0.744, 0.745, 0.746, 0.747,
0.748, 0.749, 0.75 , 0.751, 0.752, 0.753, 0.754, 0.755, 0.756,
0.757, 0.758, 0.759, 0.76 , 0.761, 0.762, 0.763, 0.764, 0.765,
0.766, 0.767, 0.768, 0.769, 0.77 , 0.771, 0.772, 0.773, 0.774,
0.775, 0.776, 0.777, 0.778, 0.779, 0.78 , 0.781, 0.782, 0.783,
0.784, 0.785, 0.786, 0.787, 0.788, 0.789, 0.79 , 0.791, 0.792,
0.793, 0.794, 0.795, 0.796, 0.797, 0.798, 0.799, 0.8  , 0.801,
0.802, 0.803, 0.804, 0.805, 0.806, 0.807, 0.808, 0.809, 0.81 ,
0.811, 0.812, 0.813, 0.814, 0.815, 0.816, 0.817, 0.818, 0.819,
0.82 , 0.821, 0.822, 0.823, 0.824, 0.825, 0.826, 0.827, 0.828,
0.829, 0.83 , 0.831, 0.832, 0.833, 0.834, 0.835, 0.836, 0.837,
0.838, 0.839, 0.84 , 0.841, 0.842, 0.843, 0.844, 0.845, 0.846,
0.847, 0.848, 0.849, 0.85 , 0.851, 0.852, 0.853, 0.854, 0.855,
0.856, 0.857, 0.858, 0.859, 0.86 , 0.861, 0.862, 0.863, 0.864,
0.865, 0.866, 0.867, 0.868, 0.869, 0.87 , 0.871, 0.872, 0.873,
0.874, 0.875, 0.876, 0.877, 0.878, 0.879, 0.88 , 0.881, 0.882,
0.883, 0.884, 0.885, 0.886, 0.887, 0.888, 0.889, 0.89 , 0.891,
0.892, 0.893, 0.894, 0.895, 0.896, 0.897, 0.898, 0.899, 0.9  ,
0.901, 0.902, 0.903, 0.904, 0.905, 0.906, 0.907, 0.908, 0.909,
0.91 , 0.911, 0.912, 0.913, 0.914, 0.915, 0.916, 0.917, 0.918,
0.919, 0.92 , 0.921, 0.922, 0.923, 0.924, 0.925, 0.926, 0.927,
0.928, 0.929, 0.93 , 0.931, 0.932, 0.933, 0.934, 0.935, 0.936,
0.937, 0.938, 0.939, 0.94 , 0.941, 0.942, 0.943, 0.944, 0.945,
0.946, 0.947, 0.948, 0.949, 0.95 , 0.951, 0.952, 0.953, 0.954,
0.955, 0.956, 0.957, 0.958, 0.959, 0.96 , 0.961, 0.962, 0.963,
0.964, 0.965, 0.966, 0.967, 0.968, 0.969, 0.97 , 0.971, 0.972,
0.973, 0.974, 0.975, 0.976, 0.977, 0.978, 0.979, 0.98 , 0.981,
0.982, 0.983, 0.984, 0.985, 0.986, 0.987, 0.988, 0.989, 0.99 ,
0.991, 0.992, 0.993, 0.994, 0.995, 0.996, 0.997, 0.998, 0.999,
1.   ])
```

Tuy nhiên, với các hàm cơ bản như trên, `Numpy` chỉ hoạt động với dữ liệu mảng. Vì vậy, khi sử dụng `Pandas` để đọc dữ liệu ở dạng DataFrame giúp kiểm tra nhanh các đặc trưng của dữ liệu, phát hiện giá trị bất thường và chuẩn bị cho các bước phân tích tiếp theo.

In [58]:
```
diamond_dataset1.describe()
```

Out[58]:

	0
count	1000.000000
mean	1.371804
std	2.211734
min	0.100475
25%	0.328469
50%	0.679692
75%	1.421732
max	19.963057

Thống kê mô tả dữ liệu catchments giúp bạn hiểu rõ hơn về phân bố, xu hướng và các đặc trưng quan trọng của dữ liệu trước khi phân tích sâu hơn. Cần lưu ý, với `.describe()` chỉ tính toán các giá trị thống kê với các **biến định lượng**.

In [59]:
```
catchments.describe()
```

Out[59]:

	springs	pH	CaMg	vegetation
count	20.0000	20.000000	20.000000	20.000000
mean	1.6500	5.000000	3.188500	50.845000
std	1.7252	0.665701	2.992542	36.434866
min	0.0000	3.800000	0.350000	3.800000
25%	0.0000	4.575000	1.175000	15.000000
50%	1.0000	5.100000	2.100000	55.000000
75%	2.2500	5.525000	4.025000	88.000000
max	6.0000	6.200000	11.000000	95.000000

In [67]:
```
np.percentile(catchments['pH'], 25)
```

Out[67]:
```
np.float64(4.574999999999999)
```

Lọc dữ liệu theo nhóm đá nền trong lưu vực (lithology) giúp phân tích riêng biệt từng nhóm, phát hiện sự khác biệt hoặc xu hướng đặc trưng giữa các nhóm trong dữ liệu.

In [60]:
```
df = catchments[catchments['lithology'] == 'basalt']
df
```

Out[60]:

	lithology	age	springs	pH	CaMg	vegetation
0	basalt	Cenozoic	0	6.2	0.35	5.8
2	basalt	Mesozoic	0	5.6	6.00	12.0
5	basalt	Mesozoic	1	5.4	0.59	12.0
7	basalt	Palaeozoic	0	5.9	2.90	6.3
13	basalt	Cenozoic	1	5.5	1.60	88.0
16	basalt	Mesozoic	0	5.7	3.40	92.0
19	basalt	Cenozoic	1	5.6	7.70	84.0

Lọc dữ liệu theo điều kiện giúp tập trung phân tích vào nhóm đối tượng có đặc điểm nổi bật, ví dụ nhóm địa chất có độ **pH > 5** và **CaMg > 2** để tìm hiểu các đặc trưng riêng biệt.

In [61]:

```
df_filttered = catchments[(catchments['pH'] > 5) & (catchments['CaMg'] > 2)]
df_filttered
```

Out[61]:

	lithology	age	springs	pH	CaMg	vegetation
2	basalt	Mesozoic	0	5.6	6.0	12.0
7	basalt	Palaeozoic	0	5.9	2.9	6.3
16	basalt	Mesozoic	0	5.7	3.4	92.0
19	basalt	Cenozoic	1	5.6	7.7	84.0

Đếm số lượng từng giá trị của nhóm đá nền trong lưu vực (lithology) giúp xác định tỷ lệ giữa các nhóm trong dữ liệu, phục vụ cho các phân tích số lượng dữ liệu thu thập cho từng nhóm hoặc kiểm tra cân bằng dữ liệu.

In [62]:

```
catchments['lithology'].value_counts()
```

Out[62]:

lithology	count
basalt	7
shale	6
sandstone	4
granite	3

dtype: int64

- `Groupby` là hàm tổng hợp tất cả các đặc trưng theo 1 đặc trưng định tính: **dataframe.groupby('column1').count()**

- Nếu muốn groupby tất cả các đặc trưng theo 2 hoặc nhiều đặc trưng định tính: **dataframe.groupby(['column_name1', 'column_name2']).count()**

- Ngoài ra, có thể lựa chọn các đặc trưng muốn groupby: **dataframe.groupby(['column_name1', 'column_name2'])['column_name3'].count()**

Ngoài hàm `count()`, chúng ta còn các hàm phổ biến như: `mean()`, `median()`, `sum()`, `min()`, `max()`, `std()`, `var()`. Đây là các hàm không tính giá trị Null.

In [63]:
```
catchments.groupby('lithology').count()
```

Out[63]:

lithology	age	springs	pH	CaMg	vegetation
basalt	7	7	7	7	7
granite	3	3	3	3	3
sandstone	4	4	4	4	4
shale	6	6	6	6	6

Nhóm dữ liệu theo nhóm đá nền trong lưu vực (lithology) giúp phân tích tổng hợp các đặc trưng của từng nhóm, phục vụ cho các nghiên cứu so sánh hoặc kiểm tra sự khác biệt giữa các nhóm.

In [64]:
```
catchments.groupby('lithology')['pH'].describe()
```

Out[64]:

lithology	count	mean	std	min	25%	50%	75%	max
basalt	7.0	5.700000	0.270801	5.4	5.550	5.6	5.800	6.2
granite	3.0	4.533333	0.115470	4.4	4.500	4.6	4.600	4.6
sandstone	4.0	5.175000	0.125831	5.0	5.150	5.2	5.225	5.3
shale	6.0	4.300000	0.389872	3.8	4.025	4.3	4.575	4.8

Nhóm dữ liệu theo biến **'lithology'** và **'age'** giúp quan sát niên đại địa chất (age) của từng nhóm đá nền trong lưu vực (lithology), đồng thời quan sát được độ phủ thực vật của từng nhóm.

In [65]:
```
catchments.groupby(['lithology', 'age'])['vegetation'].mean()
```

Out[65]:

lithology	age	vegetation
basalt	Cenozoic	59.266667
	Mesozoic	38.666667
	Palaeozoic	6.300000
granite	Mesozoic	51.000000
	Palaeozoic	70.000000
sandstone	Cenozoic	27.000000
	Mesozoic	92.000000
	Palaeozoic	91.500000
shale	Mesozoic	16.500000
	Palaeozoic	37.900000
	Precambrian	67.000000

dtype: float64

Ngoài ra, `Python` hỗ trợ sắp xếp dữ liệu DataFrame theo giá trị các cột, cần lưu ý là các giá trị phải là giá trị định lượng

In [66]:
```
catchments.sort_values(by='pH', ascending=True)
```

Out[66]:

	lithology	age	springs	pH	CaMg	vegetation
8	shale	Mesozoic	1	3.8	5.90	17.0
9	shale	Mesozoic	2	4.0	2.10	16.0
11	shale	Precambrian	2	4.1	2.10	94.0
1	granite	Mesozoic	1	4.4	11.00	28.0
4	shale	Precambrian	3	4.5	2.30	40.0
15	granite	Palaeozoic	0	4.6	1.70	70.0
18	granite	Mesozoic	0	4.6	2.20	74.0
17	shale	Palaeozoic	1	4.6	0.53	72.0
6	shale	Palaeozoic	2	4.8	8.40	3.8
10	sandstone	Palaeozoic	4	5.0	1.20	95.0
3	sandstone	Cenozoic	6	5.2	1.80	27.0
12	sandstone	Mesozoic	4	5.2	1.10	92.0
14	sandstone	Palaeozoic	4	5.3	0.90	88.0
5	basalt	Mesozoic	1	5.4	0.59	12.0
13	basalt	Cenozoic	1	5.5	1.60	88.0
2	basalt	Mesozoic	0	5.6	6.00	12.0
19	basalt	Cenozoic	1	5.6	7.70	84.0
16	basalt	Mesozoic	0	5.7	3.40	92.0
7	basalt	Palaeozoic	0	5.9	2.90	6.3
0	basalt	Cenozoic	0	6.2	0.35	5.8